MLP vs. CNN Image Classification Comparison

Matei Eugeniu Stoian

Università di Trento
mateistoian0204@gmail.com

Abstract. This study evaluates MLP, and CNN architectures across a dataset containing images of cats and dogs, which identified a linear ceiling for the MLP (62.6%), while the CNN leveraged spatial awareness to reach 77.5%.

Keywords: Binary Classification, MLP, CNN, Supervised Learning

1 Introduction

This project applies two classification algorithms—Multilayer Perceptron (MLP), and Convolutional Neural Network (CNN)—across a larger image dataset (cats vs. dogs) serving as a comparative benchmark between the linear MLP and the spatially-efficient CNN.

2 MLP Classification Model

The objective of this project is to develop a binary classification model to distinguish between images of cats and dogs. This project specifically explores the limitations of the Multilayer Perceptron (MLP) algorithm when performing computer vision tasks.

2.1 Dataset Description and Preparation

The model was trained on a dataset of 4000 images. The data was pre-split into three distinct subsets to ensure an unbiased evaluation: 2000 images for training, 1000 for validation, and 1000 for testing. Each subset maintained a balanced 50/50 split between the two classes.

Data Augmentation & Optimization The baseline tests on the initial dataset revealed a significant cat bias, where the model over-predicted the cat class to maintain a decent accuracy. To combat this and improve generalization, the following strategies were implemented:

- **Data Augmentation:** The pipeline integrated `RandomHorizontalFlip`, `RandomRotation(15°)`, and `ColorJitter` (adjusting brightness and contrast)[3] . This forces the MLP to learn that a cat is still a cat even if its pixels are slightly shifted or colored differently.

- **Normalization:** The `Normalize`[3] function was utilized to scale pixel intensities. This prevents specific pixels from dominating the weight updates due to high brightness values .
- **Adaptive Learning Rate:** A conservative initial learning rate of 0.0001 was selected to prioritize training stability. To further optimize convergence, a `ReduceLROnPlateau`[3] scheduler was implemented. It monitors the validation loss and, upon detecting a plateau (no improvement for 3 epochs), reduces the learning rate by a factor of 0.1. This adjustment mitigates gradient oscillation, allowing the optimizer to stop overshooting and settle into the deepest point of the loss function surface.

Real-time augmentations transform images before they enter the neural network. This stochastic process generates 2,000 unique variations every epoch, making the model to generalize to unseen data.

2.2 Model Description

The final architecture evolved from a three-layer network into a four-layer architecture, comprising three hidden layers and one output layer.

Theoretical Architecture The model follows an expansion-then-contraction logic. The first layer expands the input (49152 pixels) into a high-dimensional hidden space (512 neurons) to capture as many pixel correlations as possible. Subsequent layers progressively reduce the dimensionality ($512 \rightarrow 256 \rightarrow 128 \rightarrow 2$).

- **Fully Connected Layers (`nn.Linear`[3]):** These layers establish a dense connectivity pattern where every single neuron is connected to every input pixel from the input vector (49152 connections per neuron in the first layer). This forces the model to learn global relationships by calculating a weighted sum of the entire image at once.
- **Batch Normalization:** `BatchNorm1d`[3] was inserted after linear transformations. This prevents the Internal Covariate Shift (deeper layers are forced to constantly adapt to changing input distributions from earlier layers) by normalizing the inputs of each layer. Thus stabilizing the training process and allowing the model to converge more effectively.
- **ReLU Activation[1] :** $f(x) = \max(0, x)$ was chosen as the activation function after every linear transformation. Its primary role is to introduce non-linearity, enabling the stack of linear layers to approximate non-linear functions (like the shape of a dog).
- **Dropout:** `Dropout`[3] was utilized (ranging from 0.1 to 0.2). This randomly eliminates neurons during training, forcing the network to learn redundant representations so it does not rely on specific pixels or noise.

2.3 Training Procedure

Optimization was performed iteratively using the Adam algorithm. In each step, the gradients $\nabla_\theta f_t(\theta_{t-1})$ were computed via backpropagation, and the model parameters θ were updated based on adaptive moment estimation to minimize the loss function using the update rule:

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Where, θ: model weights; α: learning rate; $\widehat{m}_t, \widehat{v}_t$: bias-corrected first moment and second raw moment estimates[4] . A critical component of the procedure was the `EarlyStopping`[2] callback from PyTorch Lightning. The `val_loss` was monitored with a `patience` of 5 epochs. This prevents the model from entering the overfitting zone, where training loss continues to decrease while validation loss starts to rise. In Fig. 3. the training loss shows a characteristic "sawtooth" pattern, where spikes correspond to the model encountering difficult augmented images, followed by recovery.

2.4 Experimental Results

Throughout the project, several iterations were conducted to find the best MLP performance.

Experiment 1: Optimized MLP (Compressing input space) To mitigate the large number of trainable parameters of the MLP, the input vector was compressed by reducing image resolution ($224 \times 224 \rightarrow 64 \times 64$) and converting to grayscale. This was combined with a deeper funnel architecture ($512 \rightarrow 256 \rightarrow 128$) to force feature extraction. **Theoretical Success:** By reducing the input vector size from 150528 to 4096 elements, the model's parameter count was significantly lowered ($77M \rightarrow 2M$), preventing immediate overfitting.

Experiment 2: Deep Bottleneck MLP with Early Stopping Strategy: A 5-layer architecture, utilizing grayscale images, with 2.5M parameters and 0.4 Dropout. **Results:** Test Accuracy: 60.1%. The high Dropout caused underfitting (`train_acc` was lower than `val_acc`), suggesting the model was over-regularized.

Experiment 3: Final Configuration Strategy: A resolution of 128×128 RGB images, combined with a 4-layer architecture were found to provide the best balance. Dropout was reduced to 0.2. **Final Results: Test Accuracy:** 64.7%, **Class Balance:** 289 Cats / 358 Dogs (Correct).

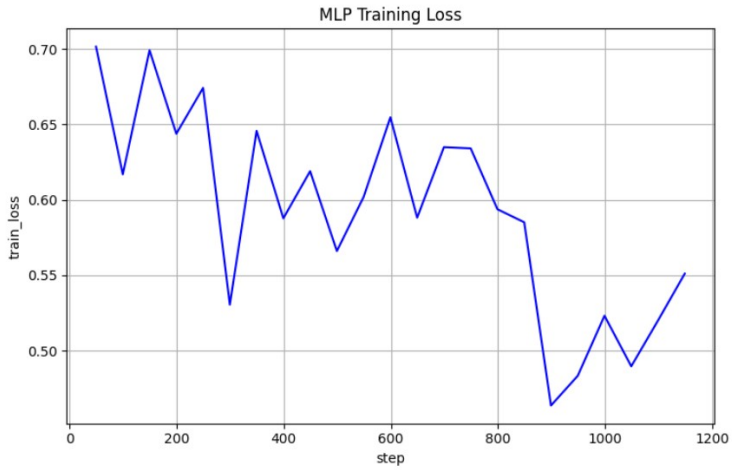

Fig. 1. Training loss variation (64.7% test accuracy case)

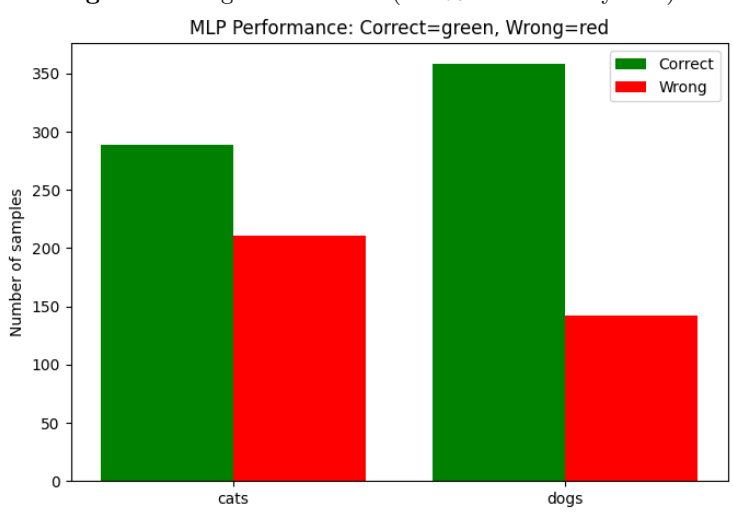

Fig. 2. Final per-class performance (64.7% test accuracy case)

2.5 Discussion: The MLP vs. CNN parallel

The experimental results identified a clear linear ceiling where the MLP struggled to exceed a mean accuracy of 62.6% due to its flattened input structure. In contrast, the implemented CNN architecture[5] – comprising a convolutional feature extractor followed by a two-layer classification head– demonstrated that preserving spatial neighborhoods is critical for visual recognition tasks.

- **Spatial Feature Extraction vs. Flattening:** While the MLP was forced to learn global patterns from a flattened vector, the CNN's convolutional

layers utilized weight-sharing filters to detect local features (e.g., edges, textures) regardless of their position. This translation invariance allowed the model to effectively cover the 14% accuracy gap, raising the mean performance to **77.5%** (peaking at 78.3%).

- **Hierarchical Abstraction via Pooling:** The inclusion of `MaxPooling2d` layers progressively decreased the spatial resolution ($128 \times 128 \rightarrow 16 \times 16$), compressing the information into high-level features before passing them to the final MLP classifier. This mechanism severely improved parameter efficiency: the CNN achieved superior results with only **8.5M parameters**, essentially one-third of the redundant **25.3M parameters** required by the fully connected baseline.
- **Convergence and Robustness:** The CNN demonstrated superior optimization, converging to a training loss of ~ 0.2 compared to the MLP's saturation at ~ 0.6. Furthermore, the convolutional approach resolved the persistent class imbalance, achieving balanced per-class accuracies of **76.2% and 80.4%**.
- **Stability Analysis:** Both architectures exhibited a consistent standard deviation of $\sim 1.3\%$ across six randomized trials, confirming that initialization variance affects both models equally. However, the CNN's mean performance maintained a robust $\sim 15\%$ lead over the MLP, proving that the architecture's ability to recognize shapes, rather than just memorize pixels, was the real reason for its superior performance.

References

1. Nicola Conci: Vision and Recognition - Lectures Notes on SVM, Classification, Features and FC NNs. University of Trento, Moodle Platform (2025-2026).
2. PyTorch Lightning, `https://lightning.ai/docs/pytorch/stable/` (2019).
3. PyTorch: `https://pytorch.org/`
4. Diederik P. Kingma, Jimmy Ba. Adam: A Method for Stochastic Optimization, `https://arxiv.org/pdf/1412.6980`.
5. Convolutional neural network: `https://en.wikipedia.org/wiki/Convolutional_neural_network`